

PhishLens (ScamShield AI)

REAL-TIME EXPLAINABLE MULTI-VECTOR SCAM INTERCEPTION ENGINE

Target SLA: Sub-1000ms Interception (< 5.0s Strict Hackathon SLA)

Project Lead: Avika Srivastava (@avksr)

Standards: GIGW 3.0 Cybersecurity, Zero-Trust Sanitization

Architecture: Parallel Multi-Agent Async Pipeline + Audit Trail

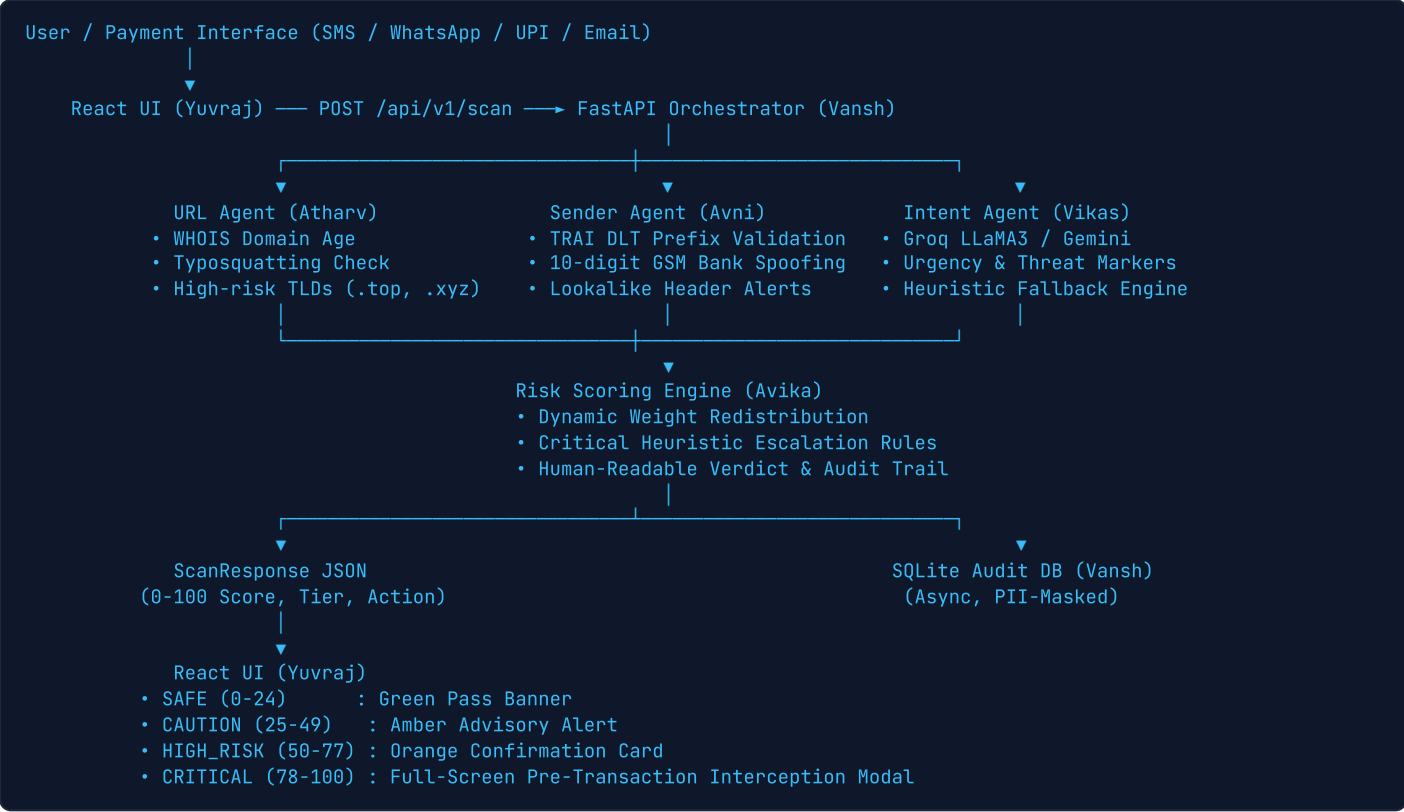
1. EXECUTIVE SUMMARY & CORE MISSION

Unlike reactive post-hoc scam registries (like Chakshu) where victims report after losing money, **PhishLens is an active, pre-transaction interception layer**. It evaluates messages, URLs, and payment requests across 3 parallel vectors to block scams *before* the user makes a payment or shares an OTP.

2. TEAM OWNERSHIP & BRANCH MATRIX

Member	Role & Module	Branch Name	Owned Files	Status
Vansh	Backend / Orchestrator	feature/vansh-orchestrator	backend/core/orchestrator.py	Pending Sprint 1
Atharv	URL & Domain Agent	feature/atharv-url-agent	backend/agents/url_agent.py	Pending Sprint 1
Avni	Sender & TRAI Agent	feature/avni-sender-agent	backend/agents/sender_agent.py	Pending Sprint 1
Vikas	LLM Intent Agent	feature/vikas-intent-agent	backend/agents/intent_agent.py	Pending Sprint 1
Avika	Risk Scoring & Synthesis	feature/avika-scoring-engine	backend/core/scoring_engine.py	Pending Sprint 1
Yuvraj	Frontend Dashboard & UI	feature/yuvraj-frontend-ui	frontend/src/*	Pending Sprint 1

3. END-TO-END PIPELINE EXECUTION FLOW



4. THE 4-TIER RISK DECISION MATRIX

Tier	Score	Action Required	Frontend UI Behavior
SAFE	0 - 24	ALLOW	Green banner confirmation. Frictionless transaction pass-through.
CAUTION	25 - 49	WARN_USER	Amber inline notice. Recommends manual verification of sender.
HIGH_RISK	50 - 77	BLOCK_TRANSACTION	Orange persistent alert card. Explicit user confirmation required.
CRITICAL	78 - 100	BLOCK_TRANSACTION	Full-Screen Hard Block Modal actively preventing payment or OTP submission.

5. INDIVIDUAL TEAMMATE ACTION CHECKLISTS



VANSI — Backend Orchestrator & DB Logging

[feature/vansi-orchestrator](#)

- **Parallel Dispatch:** Implement `run_pipeline()` in `backend/core/orchestrator.py` invoking `analyze_url()`, `analyze_sender()`, and `analyze_intent()` concurrently via `asyncio.gather()`.
- **Timeout Supervisor:** Enforce `asyncio.wait_for(timeout=3.5)` for each agent. On timeout or exception, return an `ERROR` status so the pipeline never hangs.
- **Wire Scoring:** Pass the 3 results to Avika's `compute_score(req, url_r, sender_r, intent_r)`.
- **Async SQLite Logger:** Create `backend/core/db_logger.py` using `aiosqlite` to log scans with PII masking (redacting 10-digit phone numbers and OTPs).
- **Testing:** Verify `pytest tests/test_pipeline.py -v` passes end-to-end.



ATHARV — URL & Domain Intelligence Agent

[feature/atharv-url-agent](#)

- **URL Extraction:** Implement `analyze_url(req)` in `backend/agents/url_agent.py`. If no URL in text, return `status="SKIPPED"` with score `0.0`.
- **TLD Reputation:** Check domain against `backend/data/high_risk_tlds.txt` (flag `.top`, `.xyz`, `.club`, `.cfd` → +35 risk).
- **Typosquatting Detection:** Compare extracted domain against legitimate banking domains in `backend/data/brand_domains.json` using Levenshtein distance (e.g., `sbi-kyc-verify.top` vs `onlinesbi.sbi` → +50 risk, `is_typosquatting=True`).
- **Domain Age:** Check WHOIS domain age (if age < 30 days → +40 risk).
- **Fail-Safe:** Never raise unhandled exceptions; return `status="ERROR"` on network failures.



AVNI — Sender Identity & TRAI DLT Agent

[feature/avni-sender-agent](#)

- **TRAI DLT Validation:** Implement `analyze_sender(req)` in `backend/agents/sender_agent.py`. Match sender against Indian commercial header regex `^[A-Z]{2}-[A-Z]{6}$` and verify against certified bank headers in `backend/data/trai_dlt_registry.json`.
- **Personal GSM Bank Spoofing Rule:** If sender is a 10-digit Indian phone number (`+91-9...`) AND the content claims to be a bank ("SBI", "HDFC", "ICICI", "Income Tax"), assign score `85.0` and flag `COMMERCIAL_BANK_CLAIMED_ON_PERSONAL_GSM`.
- **Lookalike Headers:** Detect unverified alphanumeric headers (e.g. `SBI-ALERT`).
- **Fail-Safe:** Return `status="ERROR"` on any unexpected failure.



VIKAS — LLM Psycholinguistic Intent Agent

[feature/vikas-intent-agent](#)

- **LLM Inference:** Implement `analyze_intent(req)` in `backend/agents/intent_agent.py` using Groq LLaMA3 / Gemini Flash with `backend/prompts/intent_prompt.txt` in deterministic JSON mode.
- **Manipulation Triggers:** Detect Panic Urgency ("within 2 hours", "blocked today"), Coercion ("arrest warrant", "police"), and Credential Harvesting ("submit PAN", "verify OTP").
- **Resilient Offline Fallback:** If API key is missing or network times out, run local regex keyword heuristics so the pipeline finishes in < 50ms without failing.



AVIKA — Risk Scoring Engine & Synthesis

[feature/avika-scoring-engine](#)

- **Dynamic Weight Normalization:** Rebalance weights (URL 40%, Sender 30%, Intent 30%) dynamically if any agent is skipped or errors.
- **Escalation Overrides:** Double Whammy (score ≥ 92), GSM Bank Spoofing (score ≥ 88), OTP Harvesting (score ≥ 90), and TRAI Whitelist override (score ≤ 12).
- **Explainable Verdicts:** Implement in `backend/core/verdict_utils.py` and verify via `test_scoring.py` (< 10ms execution).

- **React + Vite App:** Setup in `frontend/` using modern dark cyber theme (Slate `#0B0F19` , Cyan `#00F0FF` , Red `#FF3366`).
- **Scanner Input:** Message box with 1-click test presets from `datasets/payloads_high_risk.json` .
- **Risk Meter:** Animated 0–100 speedometer gauge with tier badge.
- **Audit Drawer:** Side-by-side cards showing Atharv's URL flags, Avni's sender status, and Vikas's intent tactics.
- **Interception Modal:** Pre-transaction modal triggering when `action_required == "BLOCK_TRANSACTION"` .

6. GIT PR REVIEW RULES (FOR ALL MEMBERS)

MANDATORY RULES:

- ❌ Never push directly to `main` or `dev` — always open a PR from your feature branch to `dev` .
- ❌ Never commit `.env` , `venv/` , or `node_modules/` .
- ✅ Always import shared models from `backend/shared/models.py` — never redefine Pydantic models locally.
- ✅ Your agent function must NEVER crash the pipeline — return `status="ERROR"` on any exception.
- ✅ Run `pytest tests/test_your_module.py -v` locally before requesting review from **@avksr**.